API de Colchones

1.0.0

Documentación de la API REST para el desarrollador Frontend. Estilo de referencia: Swagger UI.

Servidores

Servidor	`http://192.168.1.254:3000`

Información general

Aspecto	Detalle
URL base	`http://192.168.1.254:3000` (red local). Todas las rutas se concatenan a esta URL.
Formato	Las peticiones con cuerpo y todas las respuestas usan JSON (UTF-8).
Cabecera obligatoria (POST)	`Content-Type: application/json`
Autenticación	Ninguna. La API no requiere token ni credenciales.
CORS	Habilitado. Todas las respuestas incluyen `Access-Control-Allow-Origin: *` y las peticiones preflight `OPTIONS` responden 204, por lo que se puede consumir desde cualquier origen del navegador.
Acceso	El servidor solo es alcanzable desde equipos conectados a la misma red local (192.168.1.x).

Resumen de endpoints

Método	Ruta	Descripción
GET	/getallcolchones	Lista todos los colchones del catálogo.
POST	/buycolchon	Registra la compra de un colchón por un cliente.
GET	/getcolchonescomprados	Lista los colchones comprados por un cliente.

Colchones

Consulta del catálogo y de las compras de un cliente

GET /getallcolchones

Obtener todos los colchones

Devuelve la lista completa de colchones registrados en el catálogo (tabla **colchon**), ordenada por `id`. No requiere parámetros.

Parámetros

Este endpoint no recibe parámetros ni cuerpo.

Respuestas

200 Lista de colchones. Si no hay registros devuelve un arreglo vacío `[]`.

```
[
  {
    "id": 1,
    "nombre": "Colchon Ortopedico",
    "imagen": "https://ejemplo.com/img/ortopedico.jpg",
    "precio": 850000,
    "medidas": "140x190"
  },
  {
    "id": 2,
    "nombre": "Colchon Resortado",
    "imagen": "https://ejemplo.com/img/resortado.jpg",
    "precio": 1200000,
    "medidas": "160x200"
  }
]
```

500 Error interno al consultar la base de datos.

```
{ "error": "Error al consultar los colchones" }
```

Ejemplo con cURL

```
curl http://192.168.1.254:3000/getallcolchones
```

POST	/buycolchon	Registrar la compra de un colchón

Inserta una compra en la tabla **colchonxcliente**, asociando un cliente con un colchón. El cliente (tabla **cliente**) y el colchón (tabla **colchon**) deben existir previamente.

Cuerpo de la petición (Request body)

Tipo de contenido: `application/json`. Ambos campos son obligatorios.

Campo	Tipo	Obligatorio	Descripción
id_colchon	integer	Sí	Identificador del colchón a comprar (campo `id` de getallcolchones).
documento_cliente	integer (int64)	Sí	Número de documento del cliente que realiza la compra.

```
{
  "id_colchon": 1,
  "documento_cliente": 1083444555
}
```

Respuestas

201 Compra registrada correctamente.

```
{ "message": "El registro se insertó correctamente" }
```

400 Faltan campos o no son numéricos.

```
{ "error": "id_colchon y documento_cliente son obligatorios y deben ser numéricos" }
```

404 El cliente o el colchón no existen (violación de llave foránea).

```
{ "error": "El cliente o el colchón no existen" }
```

500 Error interno al registrar la compra.

```
{ "error": "Error al registrar la compra" }
```

Ejemplo con cURL

```
curl -X POST http://192.168.1.254:3000/buycolchon \
  -H "Content-Type: application/json" \
  -d '{"id_colchon": 1, "documento_cliente": 1083444555}'
```

GET /getcolchonescomprados — Obtener los colchones comprados por un cliente

Devuelve todos los colchones comprados por el cliente indicado. Combina la tabla **colchonxcliente** con **colchon**, por lo que cada elemento incluye los datos completos del colchón. Si el cliente no tiene compras (o no existe) responde 200 con un arreglo vacío.

Parámetros

Nombre	Ubicación	Tipo	Obligatorio	Descripción
documento_cliente	query	integer (int64)	Sí	Número de documento del cliente. Ejemplo: 1083444555

Respuestas

200 Lista de colchones comprados por el cliente. Arreglo vacío `[]` si no tiene compras.

```
[
  {
    "id_compra": 1,
    "id_colchon": 1,
    "nombre": "Colchon Ortopedico",
    "imagen": "https://ejemplo.com/img/ortopedico.jpg",
    "precio": 850000,
    "medidas": "140x190"
  },
  {
    "id_compra": 2,
    "id_colchon": 2,
    "nombre": "Colchon Resortado",
    "imagen": "https://ejemplo.com/img/resortado.jpg",
    "precio": 1200000,
    "medidas": "160x200"
  }
]
```

400 documento_cliente no fue enviado o no es numérico.

```
{ "error": "documento_cliente es obligatorio y debe ser numérico" }
```

500 Error interno al consultar la base de datos.

```
{ "error": "Error al consultar los colchones comprados" }
```

Ejemplo con cURL

```
curl "http://192.168.1.254:3000/getcolchonescomprados?documento_cliente=1083444555"
```

Nota: un mismo cliente puede comprar el mismo colchón varias veces; cada compra genera un elemento distinto, identificado por `id_compra`.

Modelos (Schemas)

Colchon

Campo	Tipo	Descripción
id	integer	Identificador único del colchón.
nombre	string (máx. 55)	Nombre del colchón.
imagen	string (máx. 255)	Ruta o URL de la imagen del colchón.
precio	integer	Precio del colchón (número entero, sin decimales).
medidas	string (máx. 25)	Medidas del colchón. Ejemplo: 140x190

ColchonComprado

Campo	Tipo	Descripción
id_compra	integer	Identificador de la compra (registro en colchonxcliente).
id_colchon	integer	Identificador del colchón comprado.
nombre	string	Nombre del colchón.
imagen	string	Ruta o URL de la imagen.
precio	integer	Precio del colchón.
medidas	string	Medidas del colchón.

CompraRequest

Campo	Tipo	Descripción
id_colchon	integer	Identificador del colchón.
documento_cliente	integer (int64)	Documento del cliente.

MensajeOk / Error

Campo	Tipo	Descripción
message	string	Presente en la respuesta 201 de /buycolchon.
error	string	Presente en las respuestas 400, 404 y 500. Texto listo para mostrar al usuario.

Códigos de estado HTTP

Código	Significado	Cuándo ocurre
200	OK	Consulta exitosa (GET).
201	Created	Compra registrada con éxito (POST /buycolchon).
204	No Content	Respuesta a la petición preflight OPTIONS (la gestiona el navegador automáticamente).
400	Bad Request	Parámetros faltantes o con formato inválido.
404	Not Found	El cliente o el colchón de la compra no existen.
500	Internal Server Error	Error inesperado en el servidor o la base de datos.

Ejemplos de consumo desde el Frontend

Los siguientes ejemplos usan **axios** (`npm install axios`). Con `fetch` el comportamiento es equivalente.

Configuración base

```
import axios from 'axios'

const api = axios.create({
  baseURL: 'http://192.168.1.254:3000',
  headers: { 'Content-Type': 'application/json' },
})
```

Listar colchones

```
async function getAllColchones() {
  const { data } = await api.get('/getallcolchones')
  return data // arreglo de colchones
}
```

Comprar un colchón

```
async function buyColchon(idColchon, documentoCliente) {
  try {
    const { data, status } = await api.post('/buycolchon', {
      id_colchon: idColchon,
      documento_cliente: documentoCliente,
    })
    return data.message // status === 201
  } catch (e) {
    // e.response.status: 400, 404 o 500
    throw new Error(e.response?.data?.error || 'Error de conexión')
  }
}
```

Consultar los colchones comprados por un cliente

```
async function getColchonesComprados(documentoCliente) {
  const { data } = await api.get('/getcolchonescomprados', {
    params: { documento_cliente: documentoCliente },
  })
  return data // arreglo de colchones comprados
}
```

Recomendaciones

- Mostrar al usuario el campo `error` de las respuestas 400, 404 y 500: ya viene redactado en español.
- Tratar el arreglo vacío `[]` como un estado válido (sin colchones o sin compras), no como un error.
- El campo `precio` es un entero: aplique el formato de moneda en el Frontend (por ejemplo, con `Intl.NumberFormat`).
- El campo `imagen` es un texto con la ruta o URL de la imagen; úselo directamente en `<img src>` si es una URL absoluta.
- Si se obtiene un error de red, verifique que el equipo esté en la misma red local y que el servidor esté activo en el puerto 3000.